HiSock: 网络跃层透传

蒲乐辉

Pu Lehui <pulehui@huawei.com>

目录

01 传统网络栈困局

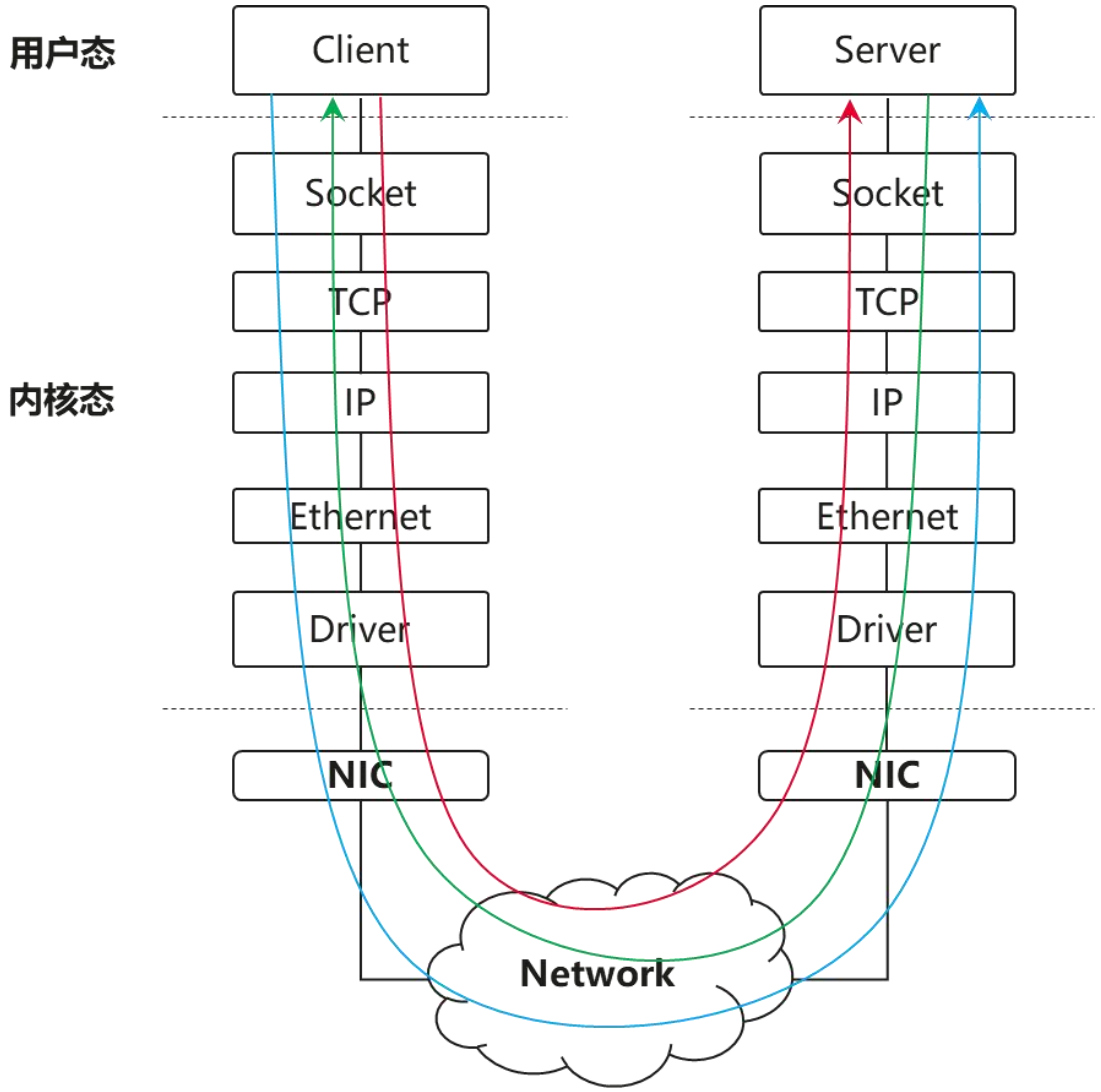
02 网络栈加速实践

03 网络跃层透传

04 性能收益

01 传统网络栈困局

Linux网络栈困局



深度抽象

五层模型层层流转，沿途 HOOK 点（过滤、连接跟踪）带来隐性开销

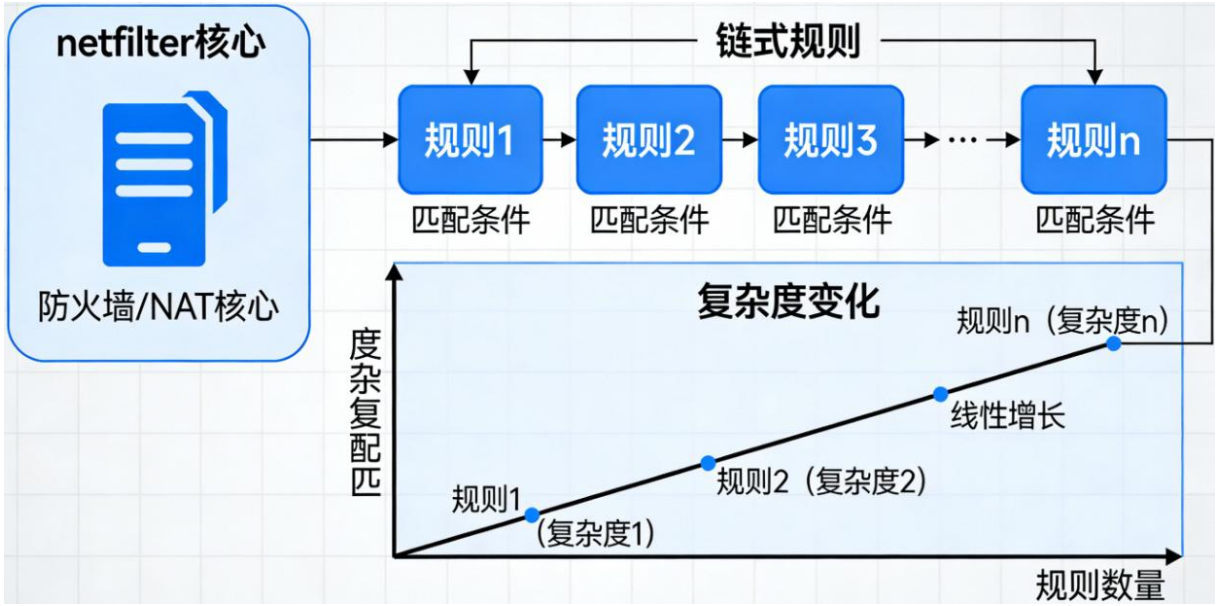
“一刀切”

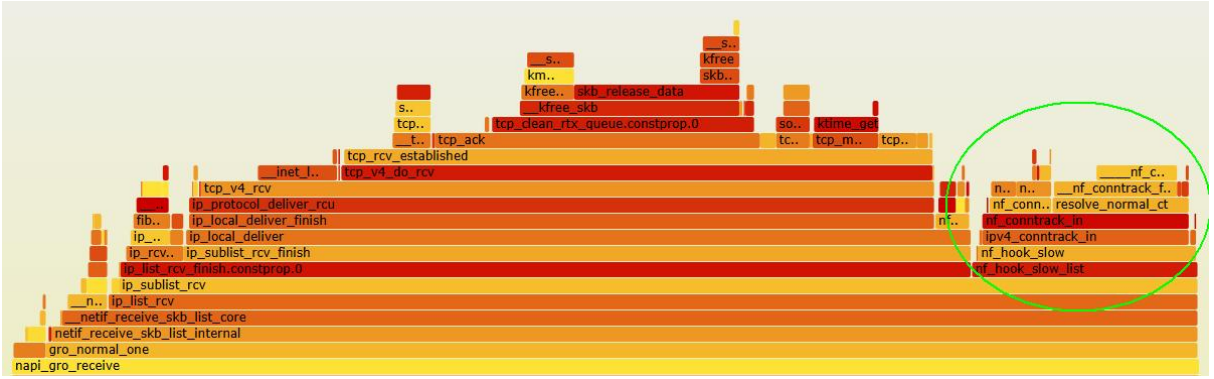
为保证 TCP/IP 兼容性，所有报文被迫遍历完整路径，无法为纯数据流做定制裁剪

算力错配

百 G 网卡时代，协议栈处理延时成为木桶短板，CPU 沦为网络 IO 的瓶颈

Netfilter复杂规则链负担

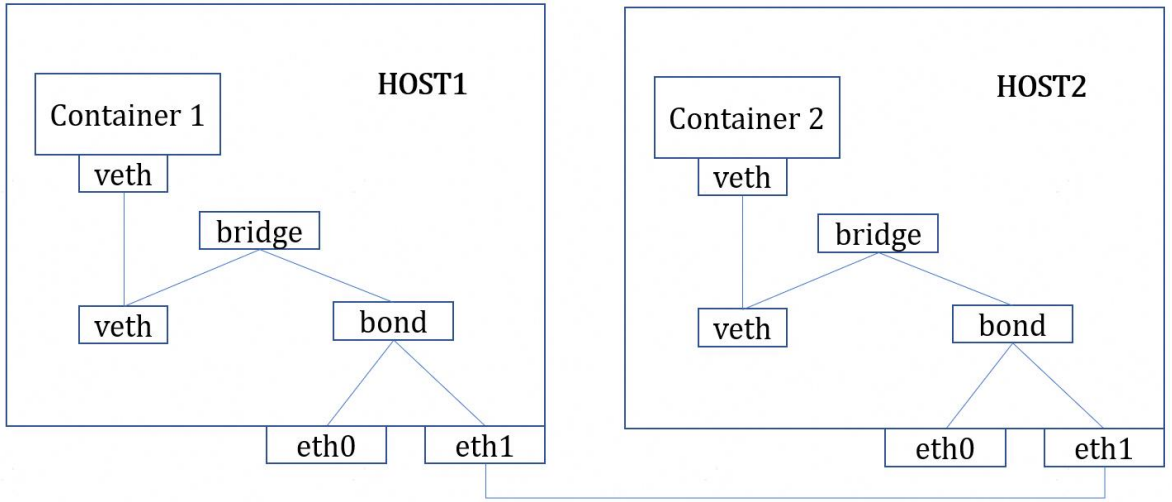


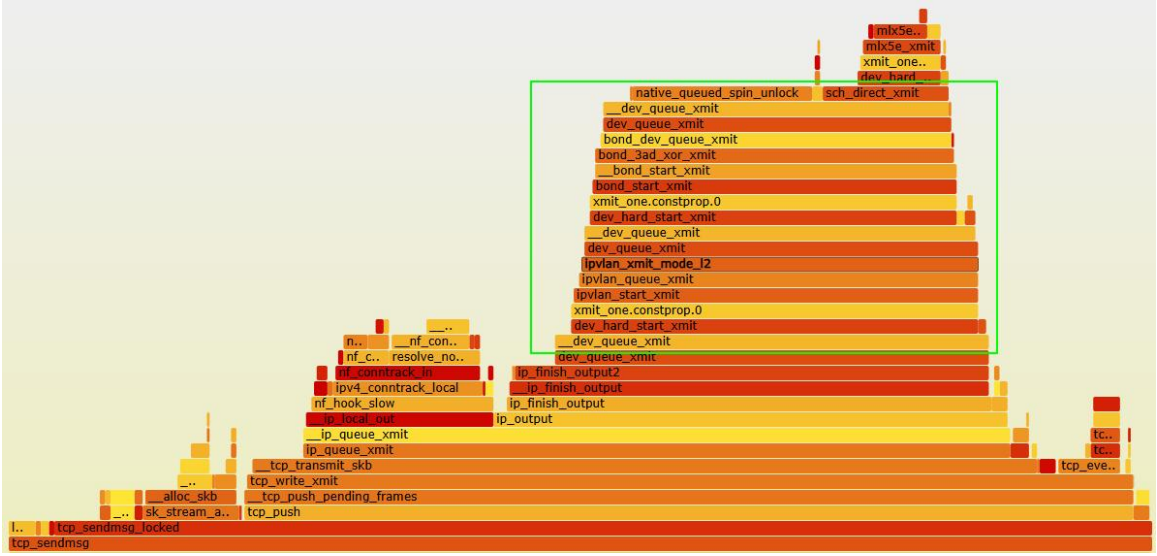


Netfilter复杂规则链

Netfilter作为防火墙和NAT核心，采用链式规则匹配机制。当规则数量增长时，匹配复杂度呈线性上升

网络设备转换负担



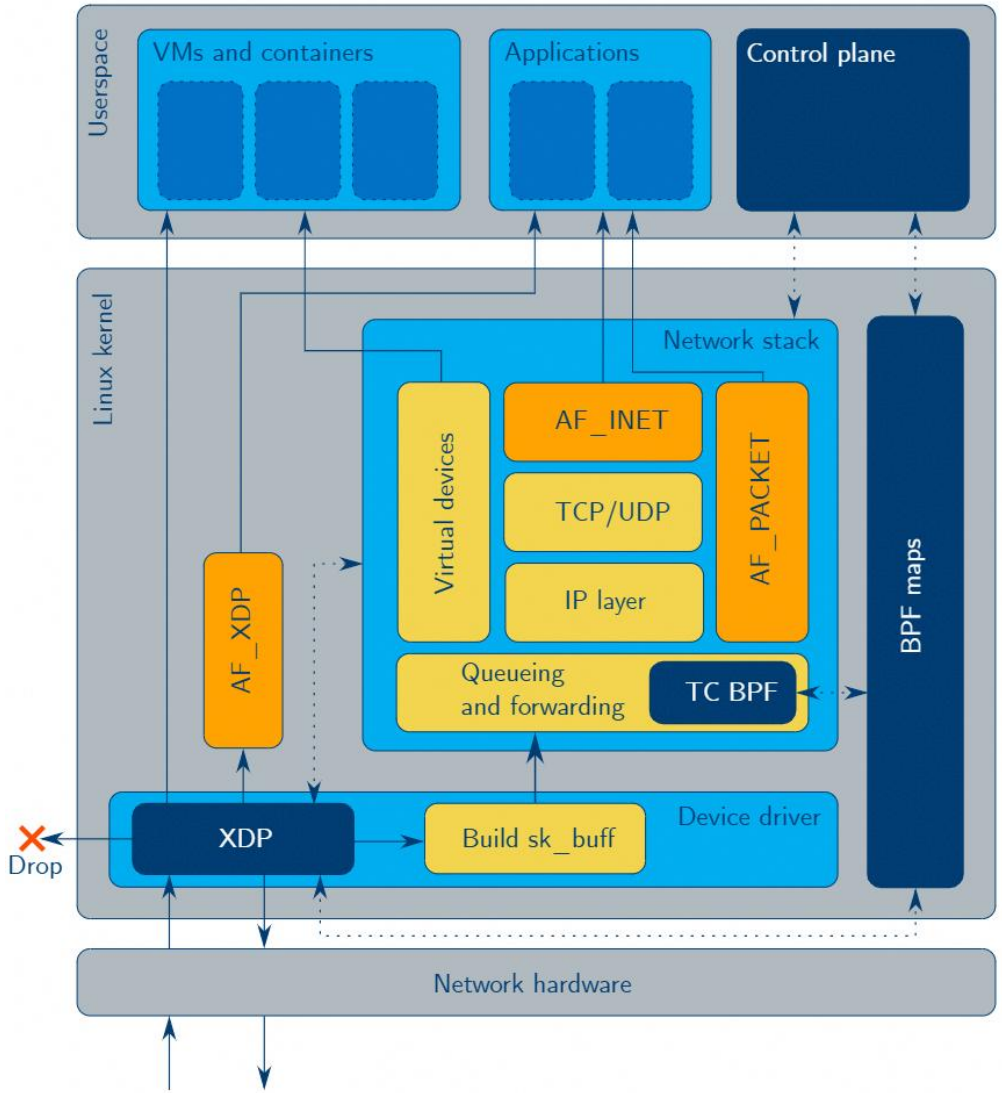


设备转换嵌套开销

容器和虚拟化场景中，veth pair、网桥、ipvlan等设备转换操作使数据包多次穿越协议栈

02 网络栈加速实践

XDP (Kernel Bypass)



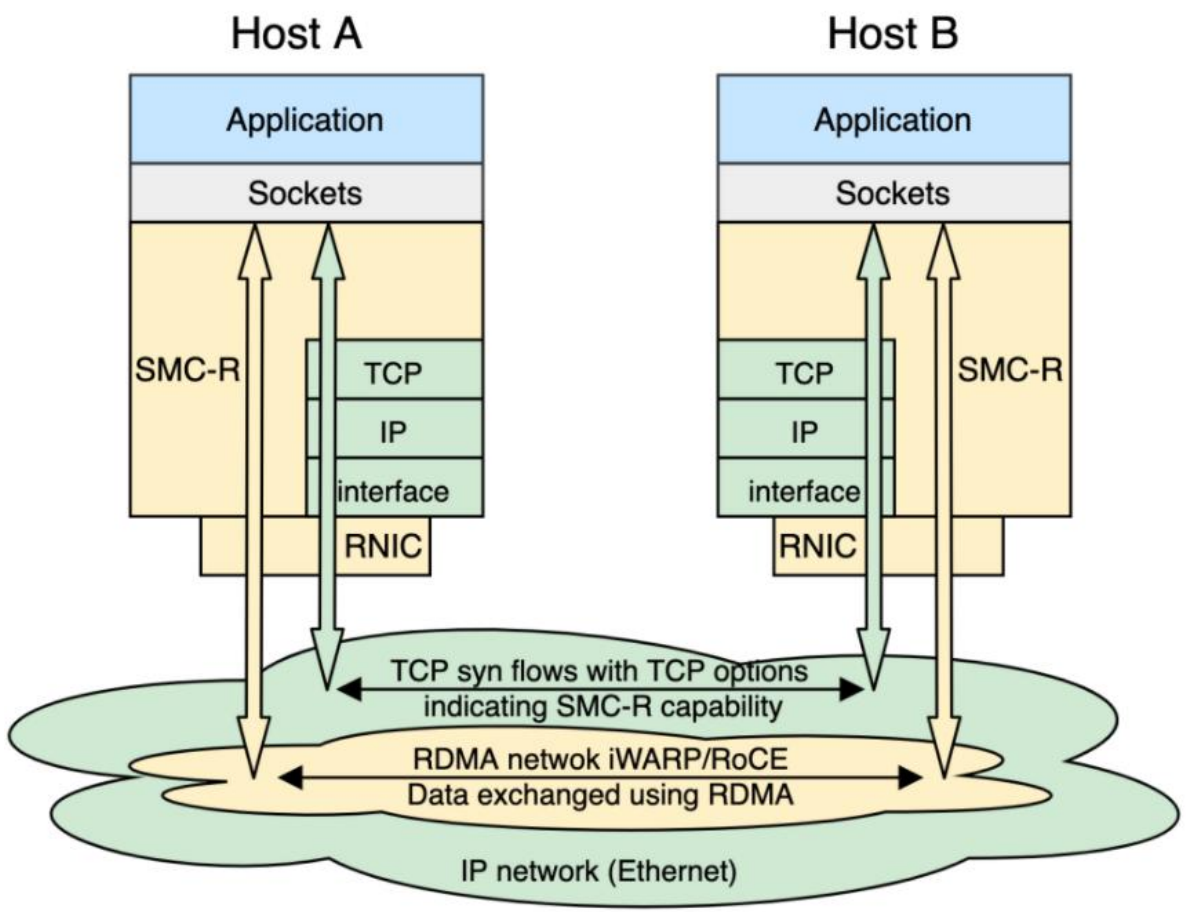
优势

Kernel-bypass，Zero-copy +Polling 将吞吐推向极致

代价

破坏 POSIX 语义，业务需侵入式改造

硬件卸载方案 (SMC-R / RDMA / DPU)



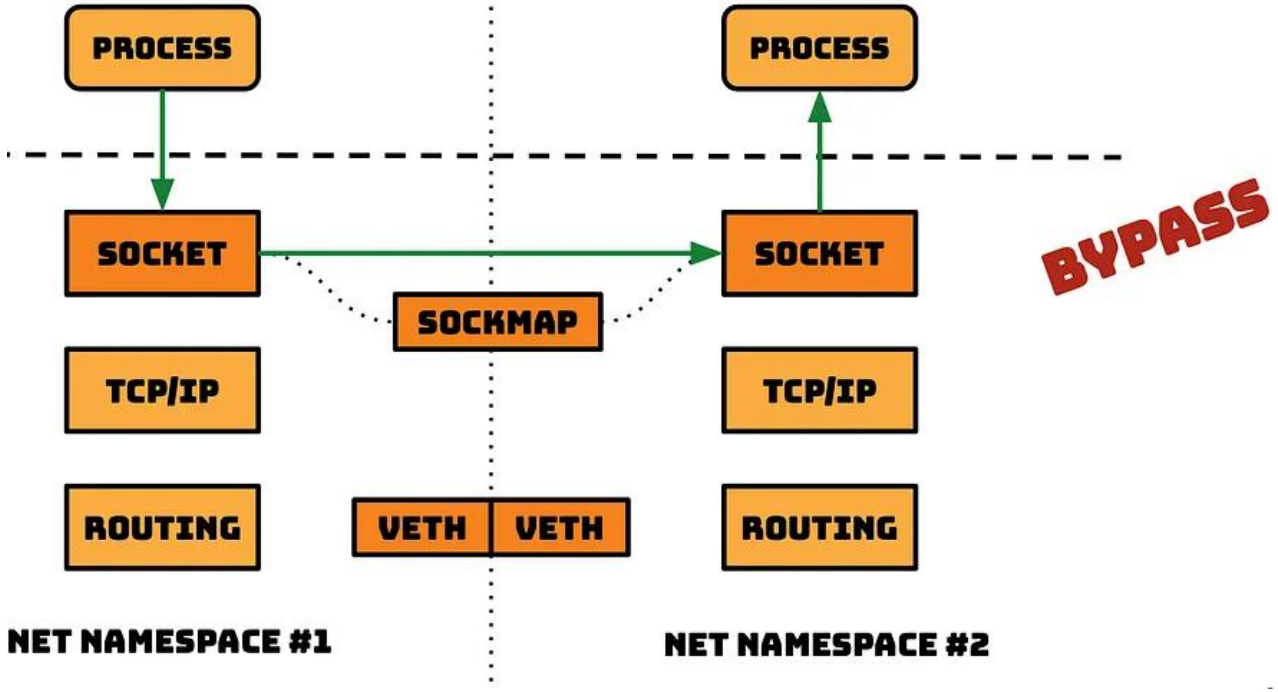
优势

协议栈开销卸载至 DPU/RDMA 硬件，彻底释放 Host CPU

代价

依赖专用网卡硬件，且通常局限在局域网内部，跨网段、跨机房互通受限

eBPF Sockmap



优势

Socket 级短接，消除本机通信的协议栈开销

代价

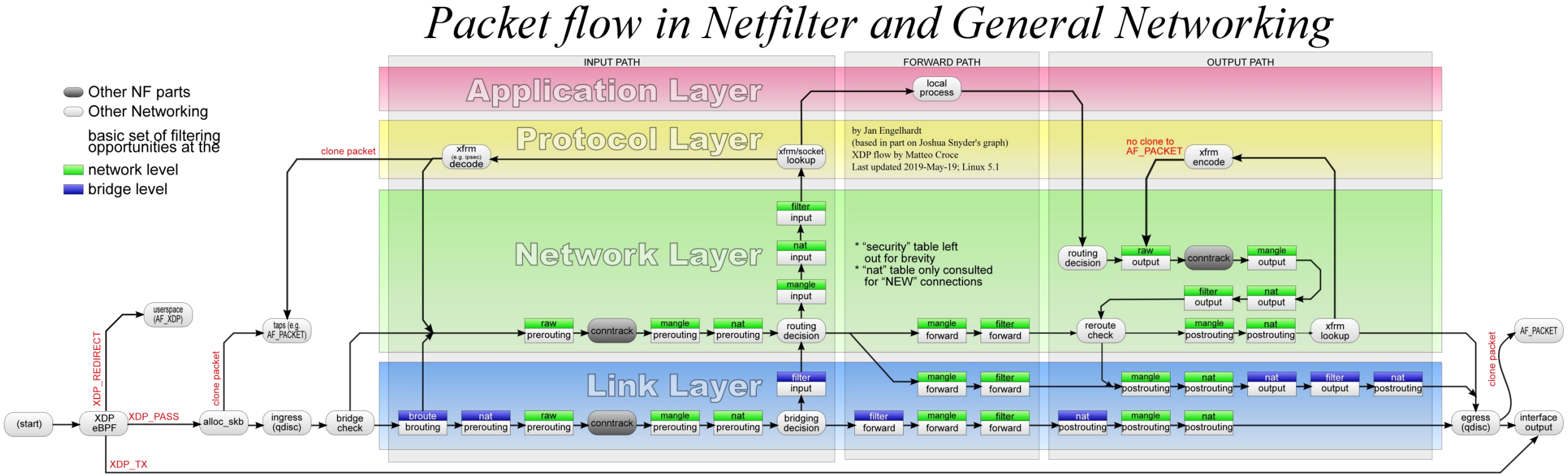
仅限本机生效，面对跨机流量束手无策

03 网络跃层透传

设计构想

消除哪些Netfilter开销?

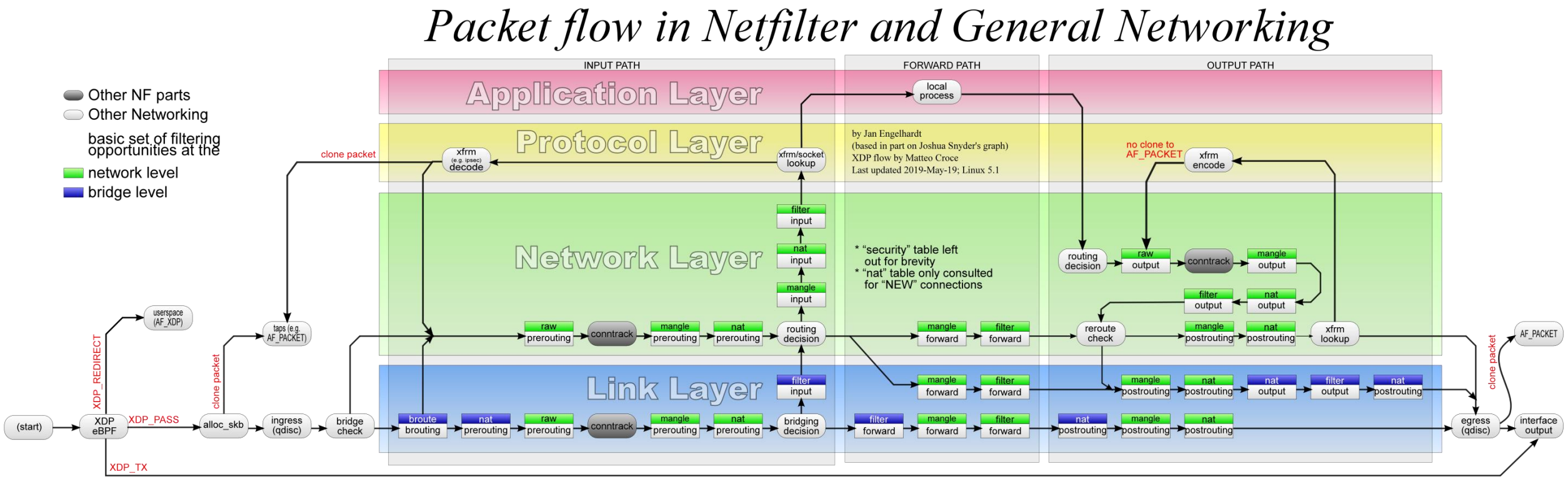
- 部分消除?
- 针对特定、信任报文进行消除?



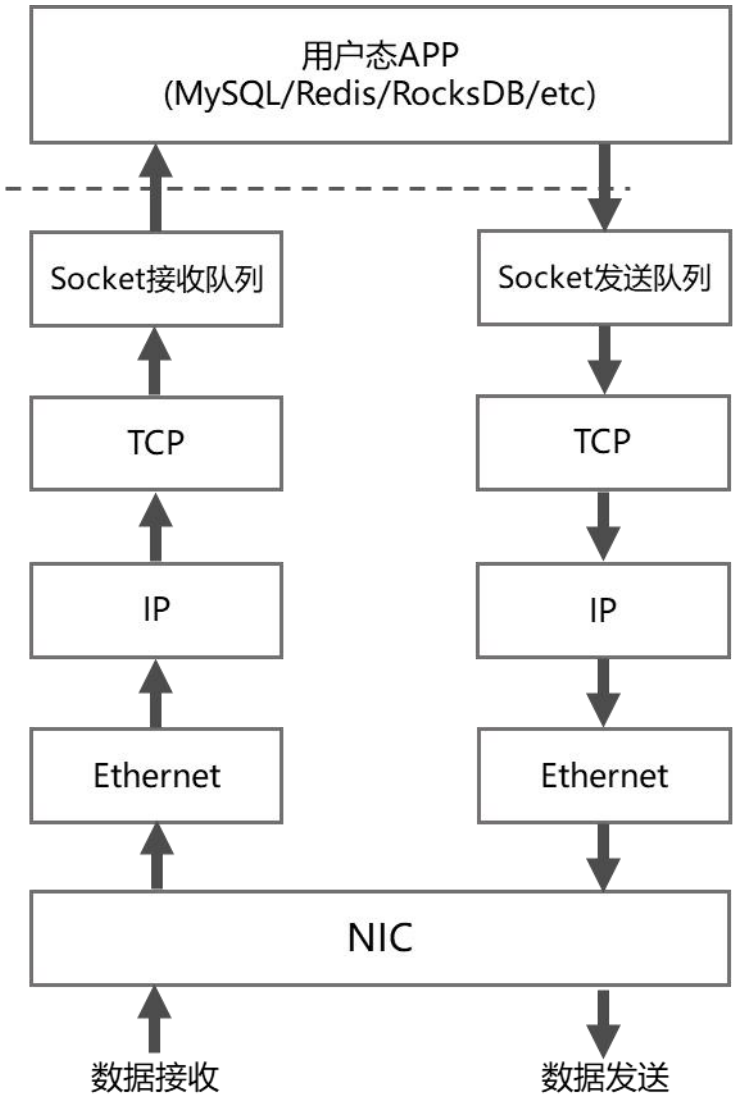
设计构想

怎么消除Netfilter开销?

- 各Hook点消除?
- 协议栈层级消除?



设计构想



挂载点

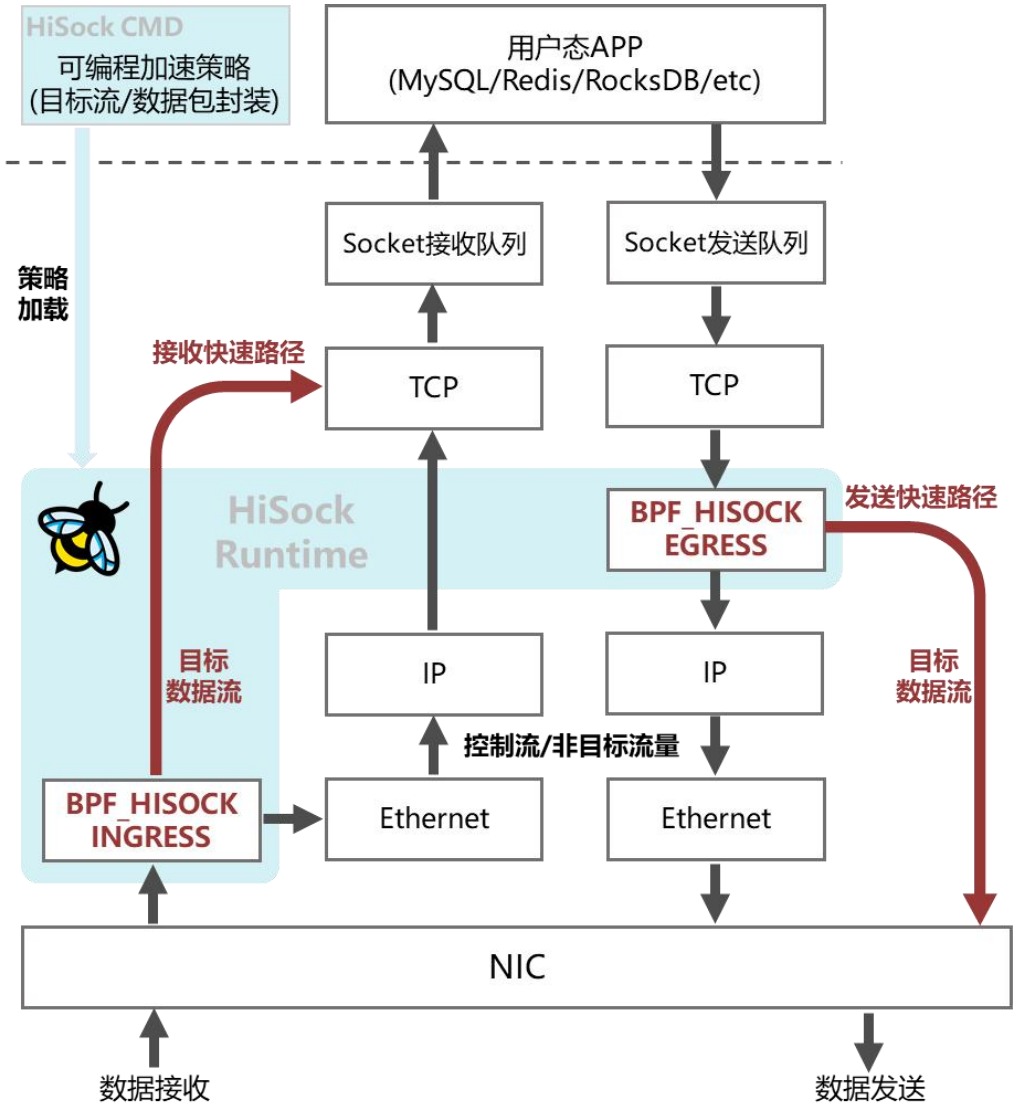
- 入口：复用XDP？协议栈入口？
- 出口：网络层入口？

BPF程序

- 统一上下文：struct __sk_buff，L3层级
- prog管理：入口 -> dev，出口 -> cgroup

```
int hisock_demo_prog(struct __sk_buff *skb)
{
        void *data_end = (void *)(long)skb->data_end;
        void *data = (void *)(long)skb->data;

        ihdr = (struct iphdr *)data;
        if (ihdr + 1 > data_end)
                return HISOCK_PASS;

        thdr = (struct tcphdr *)(ihdr + 1);
        if (thdr + 1 > data_end)
                return HISOCK_PASS;

        key.saddr = ihdr->saddr; key.sport = bpf_ntohs(thdr->source);
        key.daddr = ihdr->daddr; key.dport = bpf_ntohs(thdr->dest);

        val = bpf_map_lookup_elem(&connmap, &key);
        if (!val)
                return HISOCK_PASS;

        /* skb extend handler  */

        return HISOCK_REDIRECT;
}
```

HiSock概览



设计哲学

- “跃层”而非“重写”：复用TCP机制，叠加必要IP逻辑
- 首包学习、后续快转：针对已建链目标数据流加速转发

技术特点

用户透明、硬件无感、本跨机适用

核心组件

HiSock Runtime

- BPF_HISOCK_INGRESS/BPF_HISOCK_EGRESS
- sockops

HiSock CMD

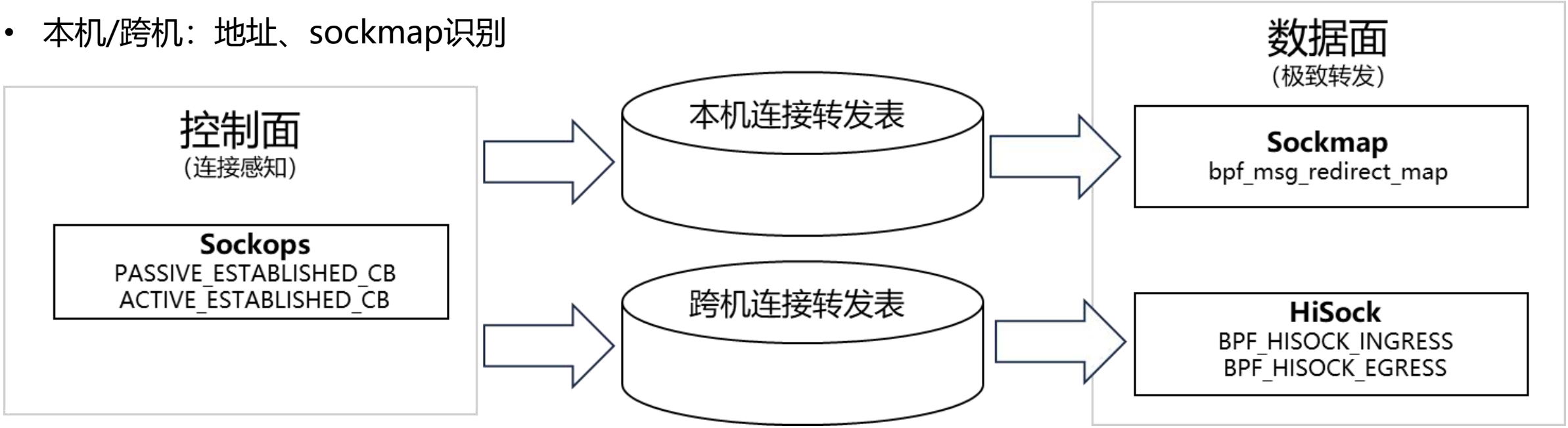
- 目标流识别
- skb扩展处理

目标流识别

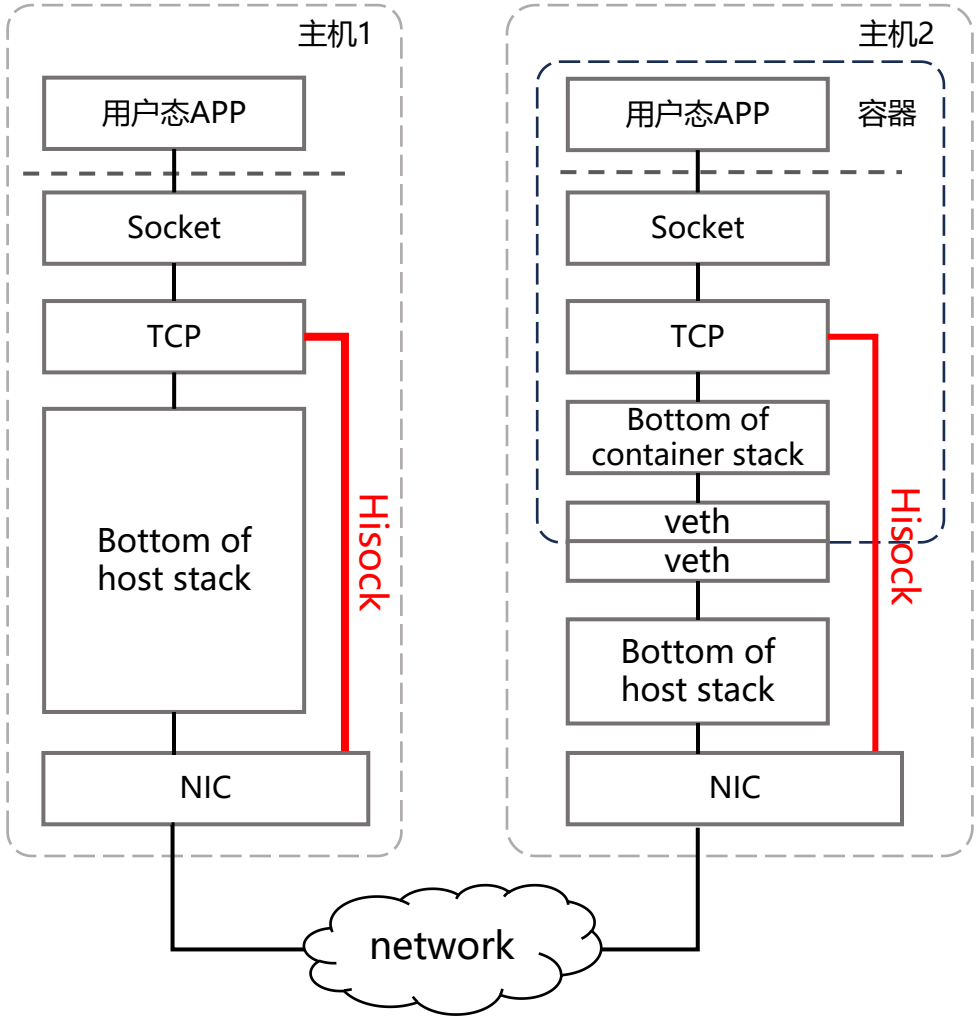
控制面与数据面协同

- 控制面：监听 TCP 握手，提取目标五元组录入转发表，精准下发策略
- 数据面：本机流量走sockmap短接，跨机流量走HiSock快速路径

- 目标流识别：端口、进程、数据包特征
- 本机/跨机：地址、sockmap识别



网络设备穿透

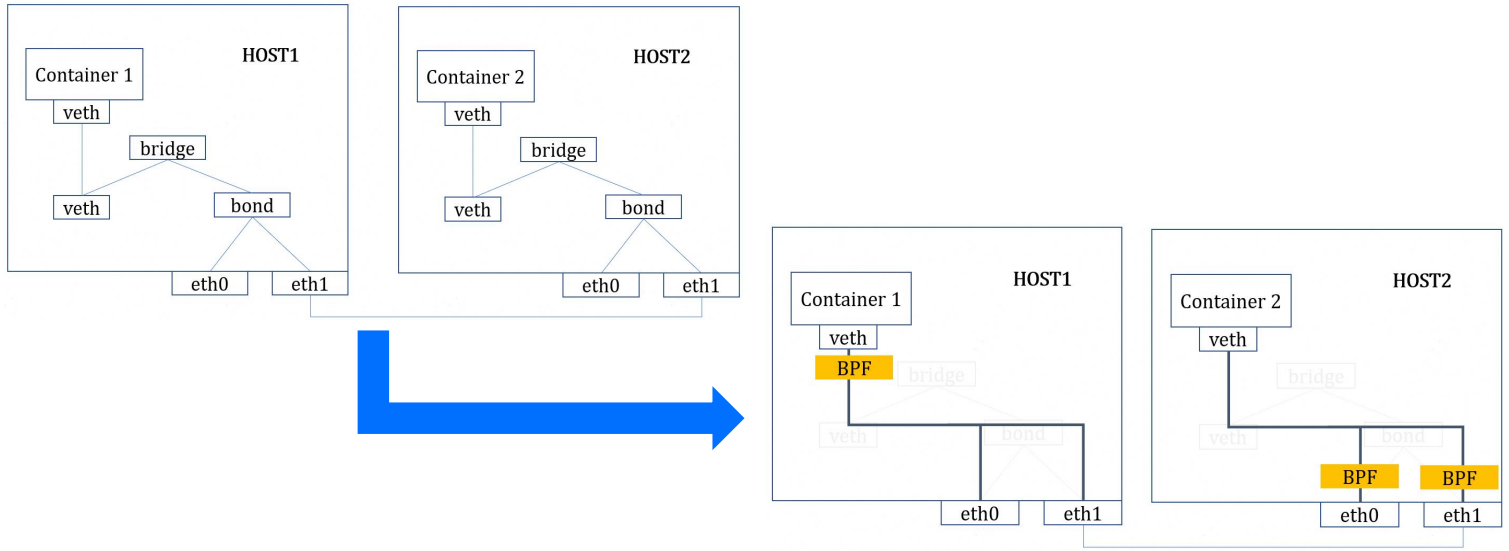


网络设备转换

- 接收路径：跟踪握手报文，最内层设备
- 发送路径：跟踪入口报文，最外层设备

关键收益

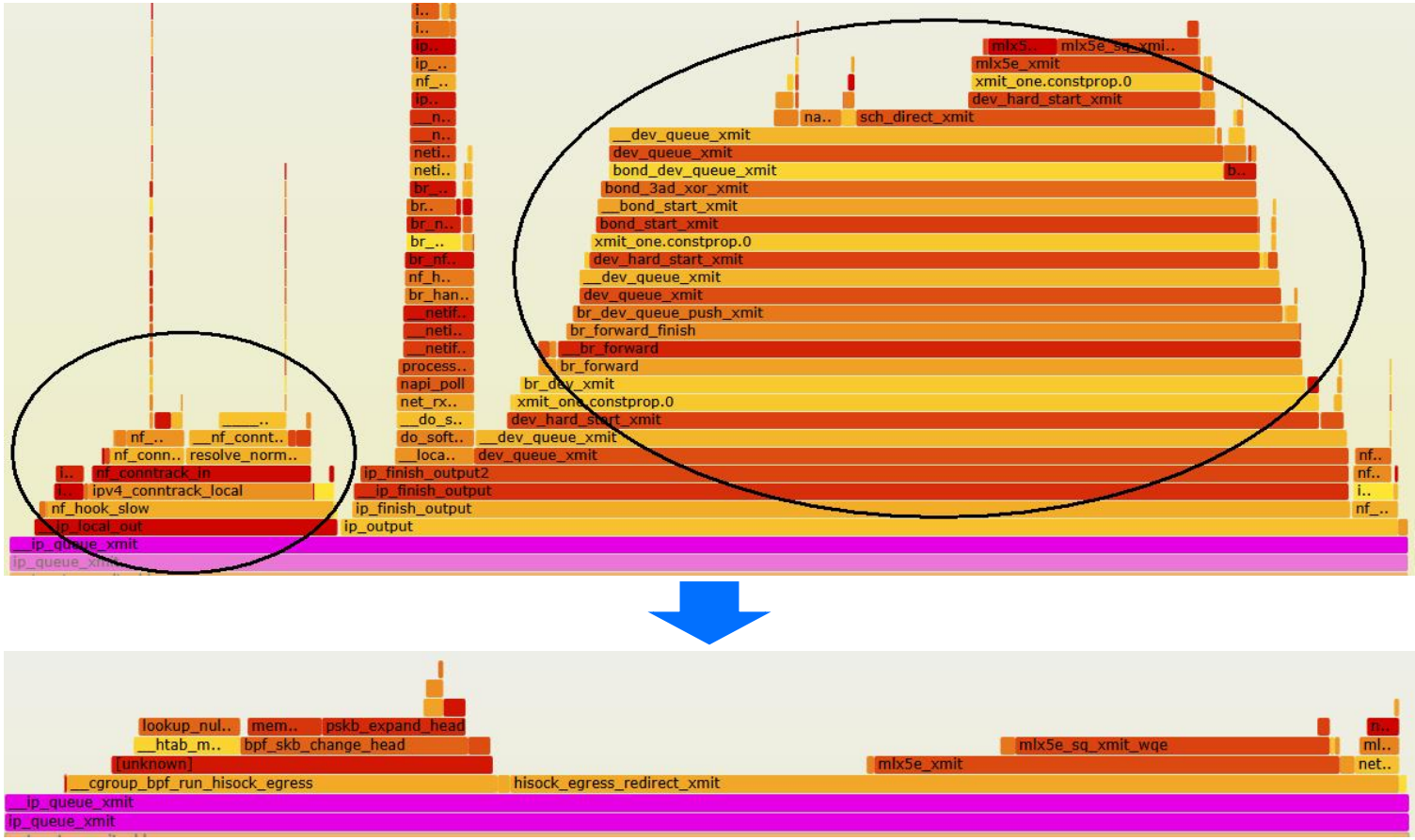
- 减少设备转换嵌套逻辑调用、路由查找开销
- 容器不经softirq直接以进程上下文发包



04 性能收益

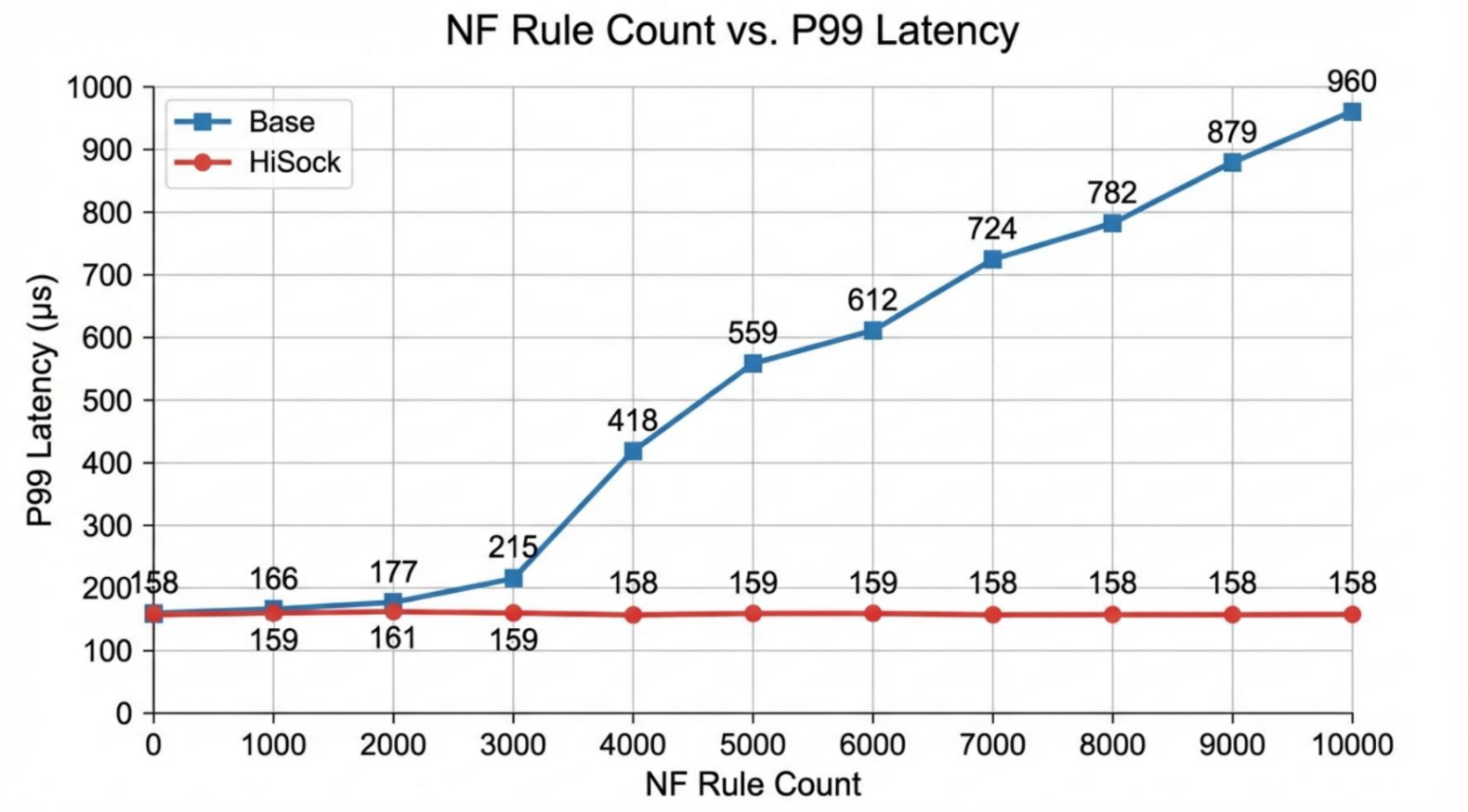
优化效果

- NF_HOOK开销消除
- 收发路径打薄，网络设备转换开销消除



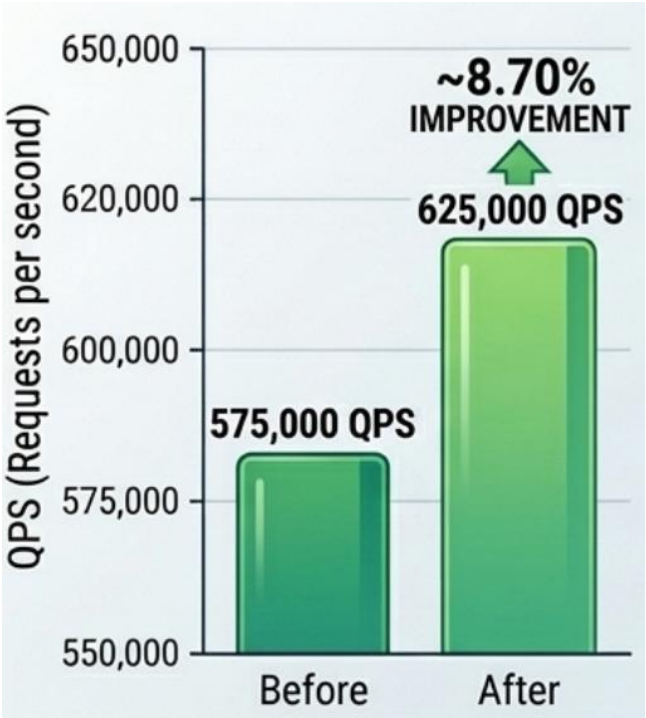
验证效果

- 传统网络栈时延随NF规则数上升而劣化，HiSock则始终保持时延稳定
- NF规则链越复杂，优化收益越大

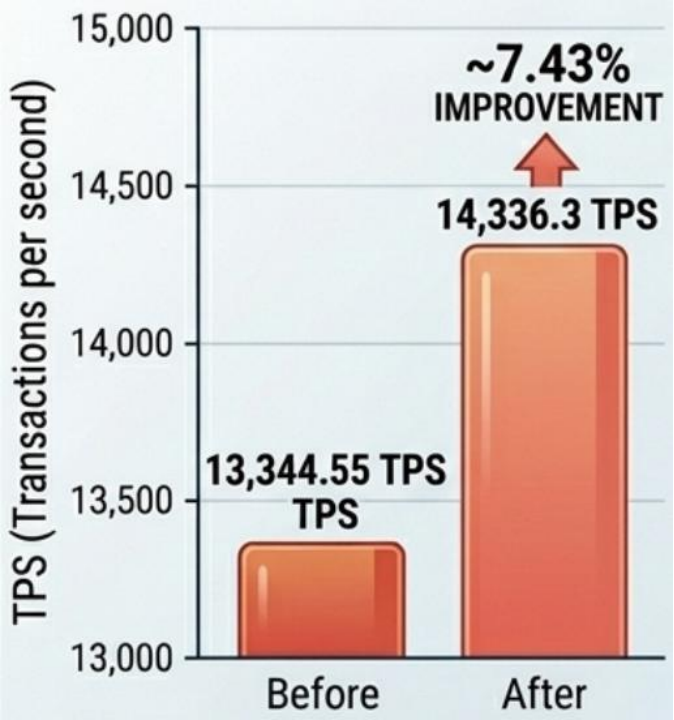


端到端收益

- 不同场景端到端收益（零NF，容器）：RocksDB 8%+，MySQL 7%+



RocksDB



MySQL

总结

- HiSock 依托“旁路加速，状态复用”设计理念，为目标数据流铺设专属快车道
- Netfilter 规则链越多，网络拓扑越复杂，优化收益越大
- openEuler：https://atomgit.com/openeuler/kernel/issues/8969

Thank you